```
// Lección 01
<h1> Introducción a Vue 3: Desarrollo Front-end </h1>

const objetivo = 'Construir tu primera Landing Page reactiva';
```

NotebookLM

El Paradigma Vue: Declarativo vs. Imperativo

Vanilla JS - Imperativo

Microgestión manual del DOM. Dices CÓMO hacer cada paso.

```
const btn = document.getElementById('btn');

btn.innerText = `Clics: ${contador}`;
```

Vue 3 - Declarativo

Renderizado reactivo. Defines QUÉ quieres ver y Vue actualiza el DOM automáticamente.

```
<button>Clics: {{ contador }}</button>
```

El HTML se conecta directamente a los datos. Si el dato cambia, la interfaz se actualiza sola.

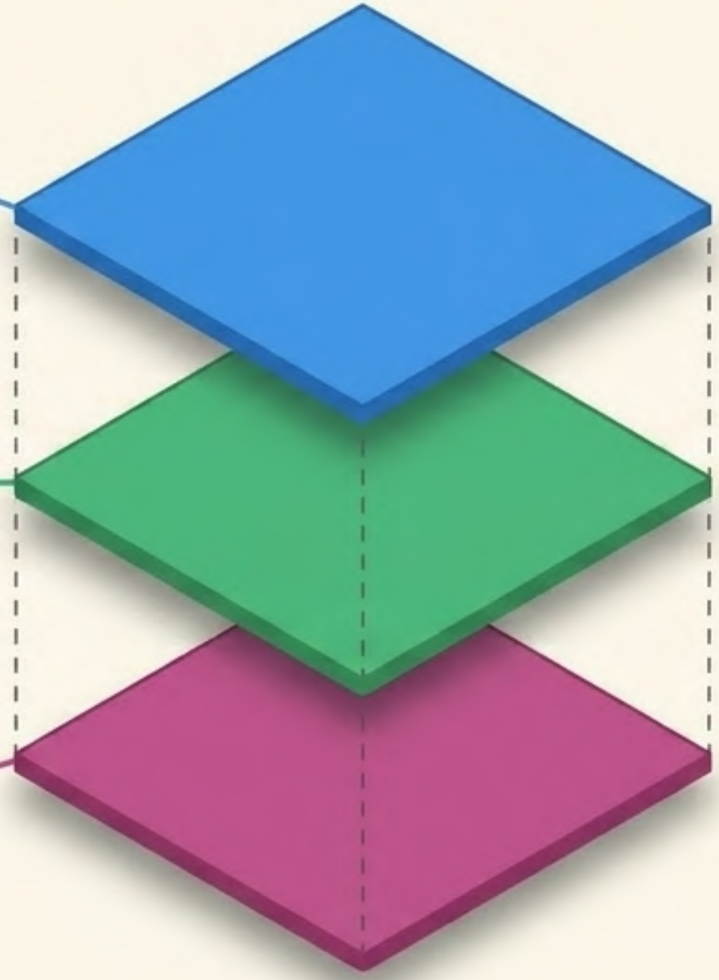
NotebookLM

Anatomía de un Componente (SFC)

```
<h1>Hola {{ nombre }}</h1>
```

```
const nombre = 'Mundo';
```

```
h1 { color: red; }
```



<template>

La estructura visual y el marcado HTML.

<script setup>

La lógica, el estado y las funciones. El atributo setup simplifica el código.

<style scoped>

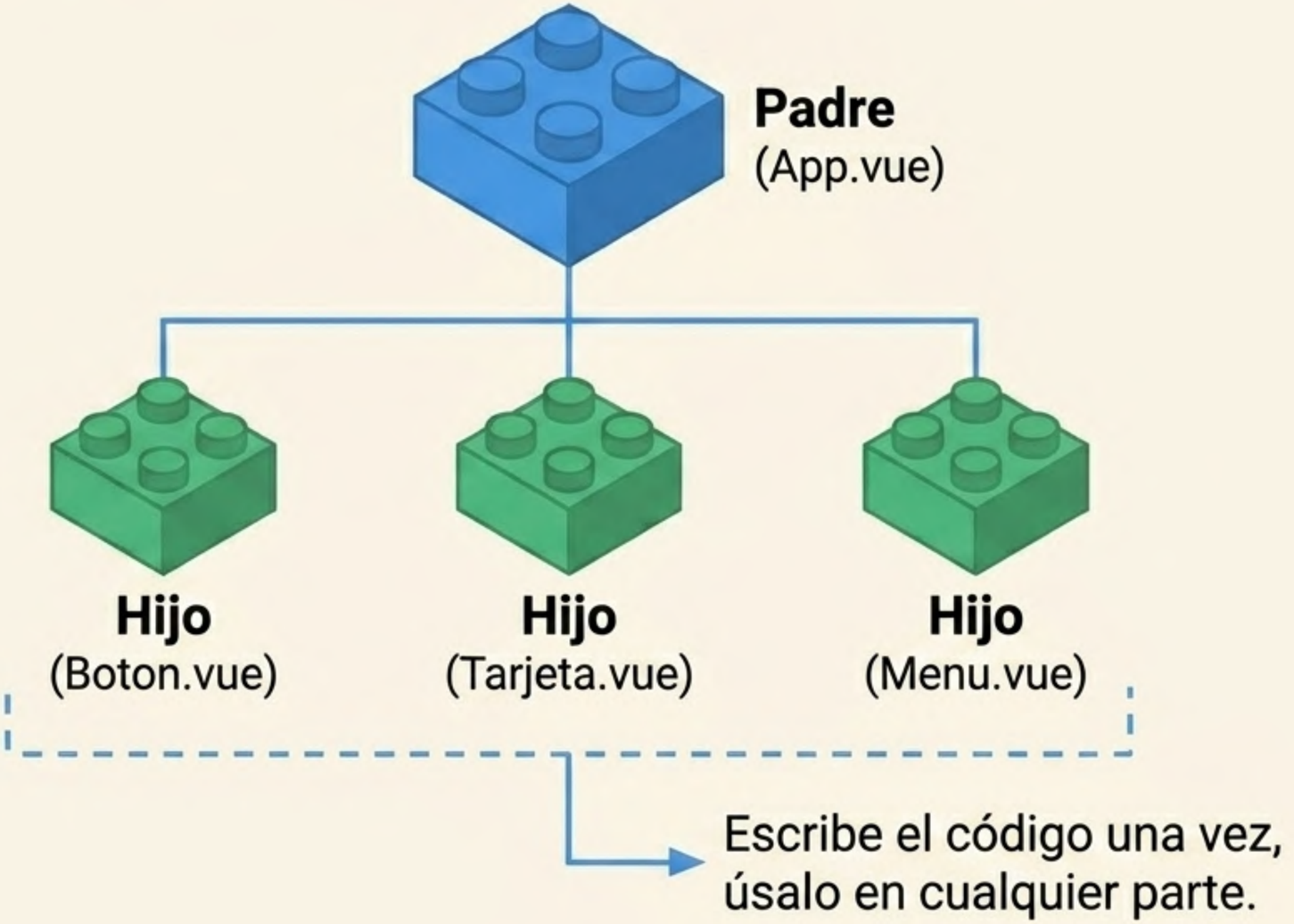
Los estilos visuales. El atributo scoped asegura que no afecten a otros componentes.

NotebookLM

¿Qué son los Componentes?

- Piezas de interfaz de usuario independientes, reutilizables y aisladas.

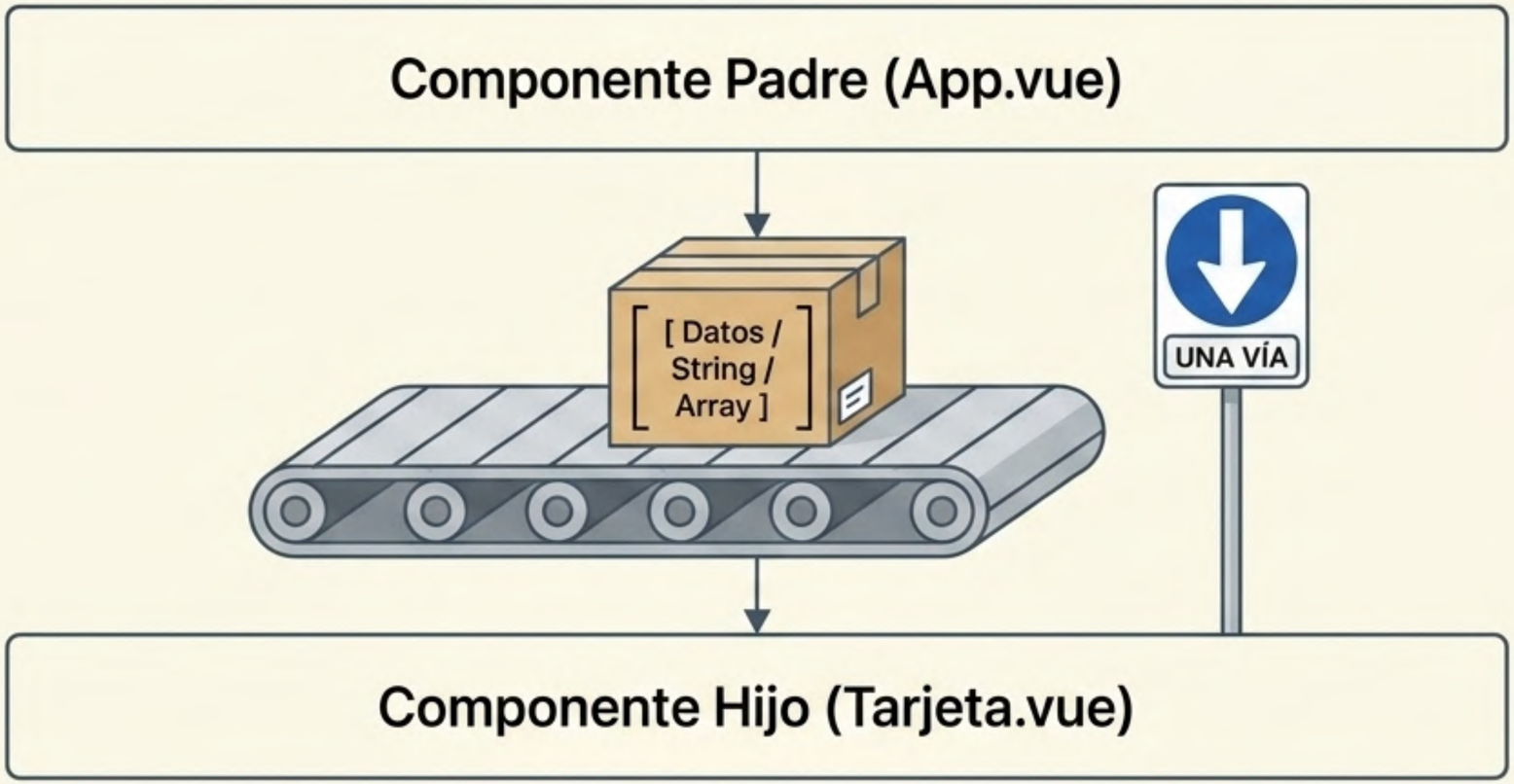
Permiten dividir una página compleja en partes pequeñas y manejables.



NotebookLM

Comunicación: ¿Qué son los Props?

Props (Propiedades) son atributos personalizados que permiten enviar datos desde un componente Padre a un componente Hijo.



Flujo Unidireccional: Los Hijos solo RECIBEN y leen Props.
Nunca deben modificar un Prop enviado por su Padre.

NotebookLM

Props en Acción

Padre.vue

```
<template>
  <SaludoTexto mensaje='¡Hola desde el Bootcamp!' />
</template>
```

El Padre inyecta el dato como un atributo HTML.

Hijo: SaludoTexto.vue

```
<script setup>
  defineProps(['mensaje']);
</script>

<template>
  <p>{{ mensaje }}</p>
</template>
```

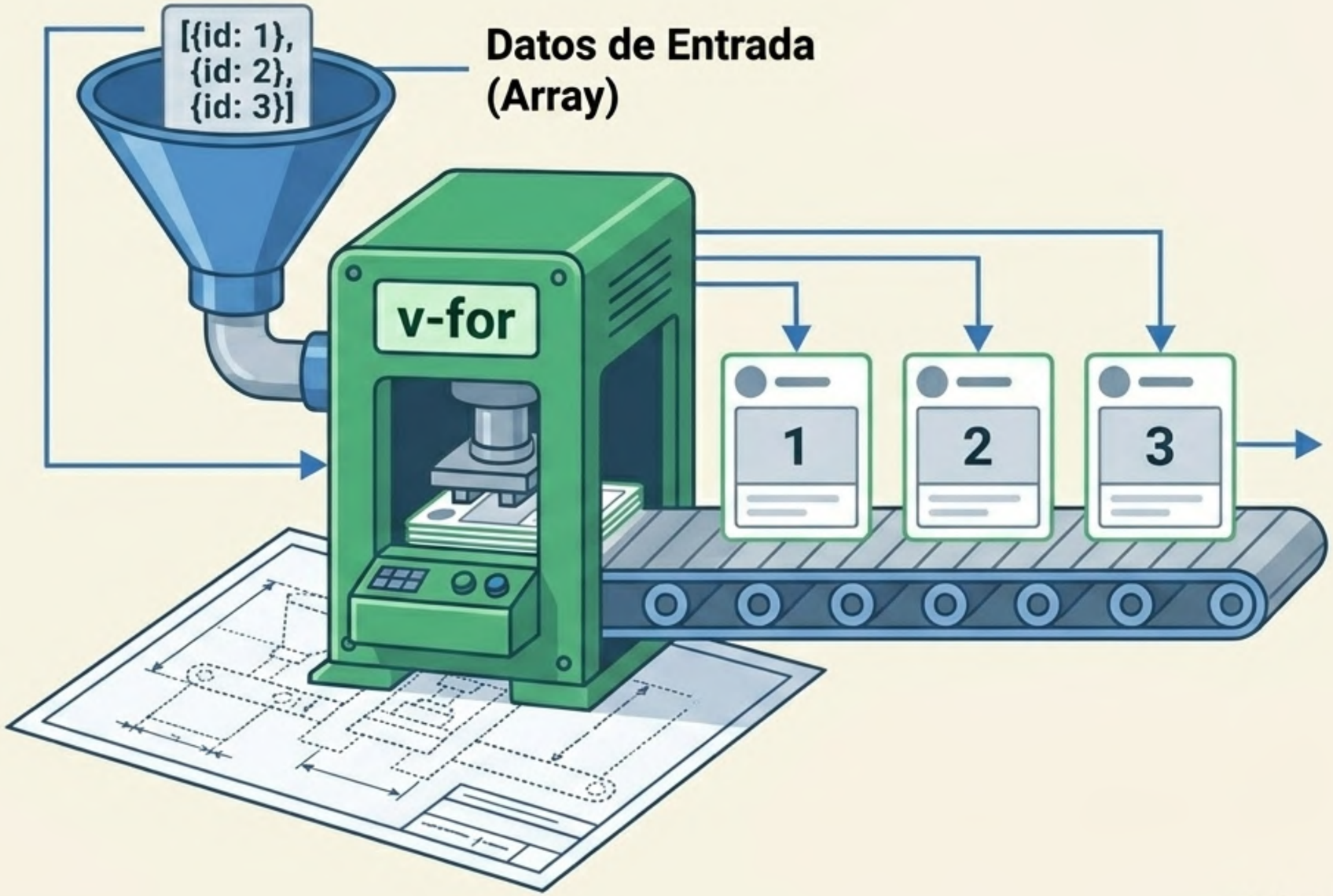
El Hijo declara qué Props acepta y los usa en su template.

NotebookLM

Renderizado de Listas (v-for)

La directiva **v-for** se utiliza para renderizar una lista de elementos (o componentes) basándose en un array de datos.

Evita copiar y pegar código repetitivo.



NotebookLM

v-for en Acción

```
<script setup>
  const lenguajes = ['JavaScript', 'Python', 'Vue'];
</script>

<template>
  <ul>
    <li v-for='(item, index) in lenguajes' :key='index'>
      {{ index }} - {{ item }}
    </li>
  </ul>
</template>
```

Itera sobre el array. “item” es el alias del elemento actual.

¡Vital! El atributo :key le da un identificador único a cada elemento. Ayuda a Vue a actualizar el DOM de forma eficiente sin re-dibujar toda la lista.

NotebookLM

Simulando Datos (JSON Local)

En el mundo real, estos datos vendrían de una base de datos o API. Aquí, los simulamos localmente.

```
<script setup>
  // Array de objetos simulando una respuesta JSON
  const servicios = [
    { id: 1, titulo: 'Diseño Web', activo: true },
    { id: 2, titulo: 'Desarrollo Frontend', activo: true },
    { id: 3, titulo: 'SEO', activo: false }
  ];
</script>
```

Estos IDs únicos serán perfectos para nuestro atributo `:key` en el bucle.

NotebookLM

Composición: Juntando las Piezas

Array de Datos (JSON) + v-for (Bucle) + Props (Envío de datos) = **Componentes Dinámicos**

En lugar de usar `v-for` en etiquetas HTML simples como **<li>**, lo usaremos directamente sobre nuestros **Componentes Personalizados**, pasándole los datos iterados a través de Props.

NotebookLM

Composición en Acción

App.vue

```
<template>
  <div class='grid'>
    <!-- Renderizando múltiples componentes dinámicamente -->
    <TarjetaServicio
      v-for='servicio in servicios'
      :key='servicio.id'
      :titulo='servicio.titulo'
    />
  </div>
</template>
```

Repite el componente <TarjetaServicio> por cada objeto en el array.

Asigna el ID único para optimización.

Usa v-bind (el atajo ':') para enviar el valor dinámico del objeto como un Prop al componente Hijo.

NotebookLM

El Ciclo de Vida: onMounted

Los componentes tienen un ciclo de vida. onMounted nos permite ejecutar código exactamente cuando el componente se inserta en el DOM.

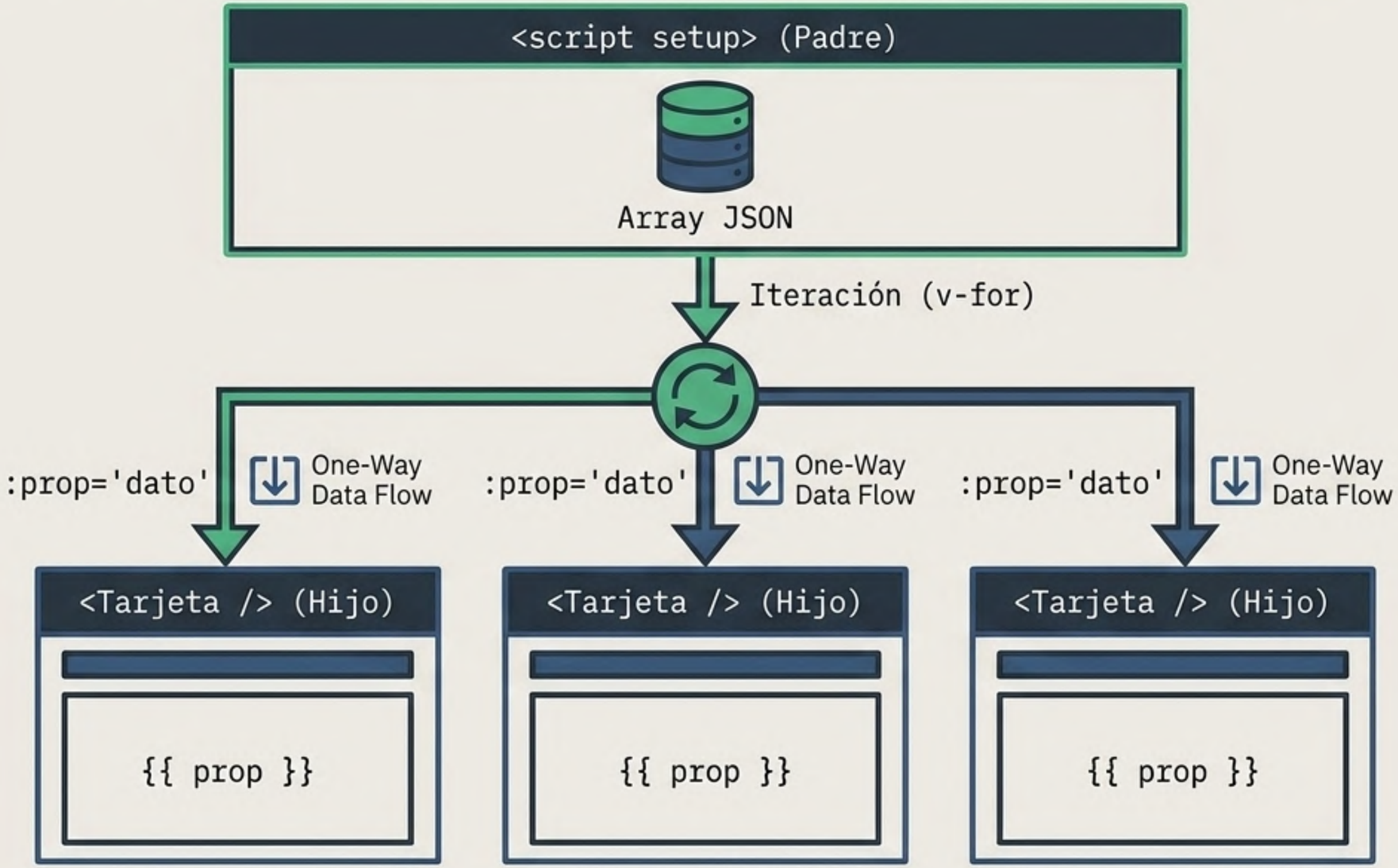
App.vue

```
<script setup>
import { onMounted } from 'vue';

onMounted(() => {
  console.log('¡El componente está listo en la pantalla!');
  // Ideal para hacer peticiones fetch() a una API
});
</script>
```

NotebookLM

Síntesis: Arquitectura de tu Landing Page



Los datos nacen en el Padre, se dividen en el bucle, y viajan hacia los Hijos para construir la interfaz

NotebookLM

Próximos Pasos: Actividad Práctica

- ☑ Paso 1: Crea un componente principal App.vue y un componente hijo Card.vue.
- ☑ Paso 2: Define un array de objetos JSON simulado en el <script setup> de App.vue.
- ☑ Paso 3: Configura Card.vue para recibir datos usando defineProps().
- ☑ Paso 4: Usa v-for en App.vue para renderizar múltiples <Card />, pasando los datos mediante atributos vinculados (:).

🚀 ¡Mantén esta presentación abierta como referencia mientras construyes! 🚀

NotebookLM